Assignment description

The task of this assignment was to design a logical data model for a simplified social media platform.

According to the assignment description, the system should support typical social media functionality.
Users create profiles with personal and location information. They can follow other users, and this relationship is directional, meaning that it does not have to be mutual.Users can create posts that may contain text, media and hashtags. Other users can interact with posts by leaving comments, reactions or sharing them.

The system should also track activity history, engagement metrics and content visibility depending on privacy settings.
An important requirement of the assignment was to design the database in a way that avoids redundant or inconsistent data.

Another requirement was to support situations where visibility, participation and engagement may change over time without deleting earlier records.
Because of this, the data model should preserve historical information while still keeping the schema clear and scalable.Based on these requirements, the logical data model was designed to represent users, relationships, posts, interactions and privacy settings in a normalized structure.

Social Media Platform

Users create profiles with personal and location information. They can follow others, but following is directional, meaning relationships are not always reciprocal. Users create posts that may include text, media, and hashtags, and other users can like, comment on, or share these posts. Activity history, engagement metrics, and content visibility based on privacy settings are tracked. The system should represent relationships, posts, interactions, and privacy considerations in a way that avoids redundant or inconsistent data.

Visibility, participation, and engagement may change over time without removing earlier records.
The structure should remain consistent as interaction patterns expand or vary.

Business description

This project models a simplified social media platform.
The goal of the database is to store users, posts and interactions between them such as comments, reactions, follows and shares.

While designing the schema I tried to separate stable data (like users and posts) from data that changes over time (such as profile information or privacy settings).
Because of that, some entities are implemented as version or history tables.

The schema is normalized to Third Normal Form (3NF) so that data duplication is minimized and relationships between entities stay clear.

Benefits of using a relational database

Using a relational database helps ensure that the data stays consistent and structured.

Foreign key constraints enforce relationships between entities such as users, posts and comments.
Normalization reduces redundancy and makes updates safer.

Another advantage is the ability to keep historical data, which is important for social media systems where profile information, privacy settings or visibility rules can change over time.

Definitions

PK – Primary Key
FK – Foreign Key
M:N – Many-to-Many relationship
3NF – Third Normal Form

Modeling decisions

During the modeling process I focused on separating core entities from historical or changing data.

For example:

- user_profile_versions stores different versions of user profile information instead of overwriting previous values.
- user_privacy_settings_history keeps track of privacy setting changes.
- post_visibility_history records how the visibility of a post changes over time.

Another design decision was implementing the many-to-many relationship between posts and hashtags using a bridge table (post_hashtags).

The follow relationship between users is implemented as a self-referencing table (follow_relationships), which represents a directed graph.

Potential anomalies prevented

The schema design helps avoid common database anomalies.

Update anomaly
Location information is stored in the locations table instead of repeating city and country fields in multiple profile records.

Insertion anomaly
Hashtags can be created independently before being linked to posts.

Deletion anomaly
Historical records such as visibility settings or profile versions are preserved instead of being overwritten.

Scalability considerations

The schema is designed to remain flexible as the platform grows.

For example:

- new event types can be added through the activity_events table
- reaction types can be extended in the reactions table
- engagement statistics are stored in engagement_metrics_daily to allow historical analytics

This approach makes it possible to expand interaction types without changing the core structure of users and posts.

Personal reflection

While designing the model, the most challenging part was deciding how to store historical information without making the schema too complex.
Using version tables for profiles and visibility settings seemed like a good compromise between flexibility and clarity.

Another interesting aspect was modeling the follow relationships between users, since it required a self-referencing relationship.